

Membros Estáticos em Java

Dani Juliano Czelusniak, Prof. Dr.

Atualizado em 2026

1 Análise Teórica e Contextualização Pedagógica

O paradigma de Programação Orientada a Objetos (POO) em Java permite a criação de estruturas robustas por meio de classes e objetos. No entanto, existem cenários onde atributos e métodos pertencem à classe como um todo, e não a instâncias específicas (objetos). Tais membros são chamados de **membros estáticos**. Este material aborda a teoria e a prática dos membros estáticos, finalizando com a implementação completa de uma Calculadora.

2 Membros Estáticos

Membros estáticos (marcados com a palavra-chave `static`) são acessados diretamente através da classe, sem a necessidade de instanciar um objeto.

Um exemplo clássico na linguagem Java é a classe `String`. O método `format` é estático, sendo chamado como `String.format("...")`. O alvo da chamada é a própria classe. Em contraste, o método `toUpperCase()` não é estático, e deve ser chamado a partir de um objeto instanciado, como `frase.toUpperCase()`.

2.1 Exemplo Prático em Código Fonte

Abaixo apresentamos um código Java completo que demonstra essa diferença estrutural.

```
1 public class ExemploMetodosString {
2
3     public static void main(String[] args) {
4
5         System.out.println("--- 1. Exemplo de Metodo Estatico ---");
6         /*
7          * O metodo 'format' pertence a CLASSE String.
8          * Nao precisamos criar um objeto String antes para usa-lo.
9          * Ele atua como uma ferramenta utilitaria que devolve uma
10        nova String formatada.
11        */
12        String nome = "Maria";
13        int idade = 25;
14
15        // A chamada e feita diretamente: NomeDaClasse.nomeDoMetodo()
```

```

15     String textoFormatado = String.format("A aluna %s tem %d anos
16     .", nome, idade);
17     System.out.println(textoFormatado);
18
19     System.out.println("\n--- 2. Exemplo de Metodo de Instancia (
20     Nao estatico) ---");
21     /*
22     * O metodo 'toUpperCase' pertence ao OBJETO String.
23     * Para usa-lo, precisamos de uma instancia real na memoria,
24     * pois ele precisa saber "qual" texto ele deve transformar
25     em maiusculo.
26     */
27     String frase = "estudando programacao orientada a objetos";
28
29     // A chamada e feita a partir da variavel (objeto):
30     nomeDoObjeto.nomeDoMetodo()
31     String fraseMaiuscula = frase.toUpperCase();
32     System.out.println(fraseMaiuscula);
33
34     // Tentar fazer String.toUpperCase() geraria um erro de
35     compilacao,
36     // pois a classe em si nao tem um texto especifico para
37     converter.
38 }

```

Listing 1: Diferença entre métodos estáticos e de instância

2.2 Resumo da Execução

- **String.format(...)**: O alvo da chamada é a classe **String**. O compilador entende isso perfeitamente porque o método foi definido com a palavra-chave **static**. Ele não depende do estado de nenhum objeto criado na memória.
- **frase.toUpperCase()**: O alvo da chamada é o objeto referenciado pela variável **frase**. Como ele não possui a palavra-chave **static** na sua definição, a linguagem exige que o programador especifique sobre qual texto particular a ação deve ser aplicada.

2.3 Atributos Estáticos

Atributos estáticos são compartilhados por todas as instâncias da classe. Se um método ou atributo for declarado como **private static**, ele será acessível apenas pelos métodos da própria classe, promovendo o encapsulamento.

```

1 class SomatoriaCoordenadas {
2     // Atributo estatico privado: acessivel por todos os metodos da
3     classe
4     private static double tx = 0;
5     private static double ty = 0;

```

```

5
6 // Metodo estatico privado: pode ser chamado internamente
7 private static double polinomio(double x) {
8     tx += x; // Acesso ao atributo da classe
9     final int a = 3, b = -8, c = 1, d = 4; // Variaveis locais
10    return (a * Math.pow(x, 3) + b * Math.pow(x, 2) + c * x + d);
11 }
12
13 public static void main(String[] args) {
14     for (double x = 0; x <= 4; x++) {
15         double y = polinomio(x); // Chamada de metodo da classe
16         ty += y; // Acesso a variavel local e de classe
17         System.out.println("(" + x + ", " + y + ")");
18     }
19     System.out.println("tx = " + tx);
20     System.out.println("ty = " + ty);
21 }
22 }

```

Listing 2: Exemplo de Membros Estáticos - Somatória de Coordenadas

No exemplo acima, destacam-se:

- **Atributo da classe:** tx e ty, que guardam o estado global para a classe e podem ser modificados internamente.
- **Variáveis locais e Parâmetros:** Como a, b, c, d e x, são acessíveis apenas no escopo do próprio método em execução.

3 Estudo de Caso: Calculadora Console

A construção de um sistema de calculadora via console demonstra de forma clara o uso de métodos estáticos públicos e privados. O sistema é estruturado em partes contendo a classe lógica (Calculadora) e a classe de interface (CalculadoraConsole).

3.1 Métodos Estáticos Públicos

A classe Calculadora expõe operações matemáticas que não dependem de estado interno, logo, seus métodos podem ser estáticos e públicos para servirem como utilitários acessíveis de qualquer parte do projeto.

3.2 Interface Homem-Computador

A classe CalculadoraConsole utiliza a classe Scanner instanciada estaticamente para interagir com o usuário, exibindo um menu, validando entradas por métodos privados estáticos (como ler_int e ler_real) e consumindo a classe lógica para processar o cálculo, onde apenas o método main necessita ser público para servir de ponto de entrada.

4 Código-Fonte Completo

Abaixo apresentamos a integração do sistema de Calculadora, contemplando as rotinas de leitura de dados e menu de opções apresentadas no estudo de caso.

```
1 import java.util.Scanner;
2
3 // Classe Logica (Pode estar em um arquivo separado chamado
4 // Calculadora.java)
5 class Calculadora {
6     public static double somar(double a, double b) { return a + b; }
7     public static double subtrair(double a, double b) { return a - b; }
8     public static double multiplicar(double a, double b) { return a *
9     b; }
10    public static double dividir(double a, double b) { return a / b;
11    }
12 }
13
14 // Classe de Interface com o Usuario (Pode estar em
15 // CalculadoraConsole.java)
16 public class Main {
17     // Atributo estatico privado
18     private static Scanner teclado = new Scanner(System.in);
19
20     // Metodo estatico privado para ler inteiro com validacao
21     private static int ler_int(int minimo, int maximo) {
22         int valor_lido;
23         do {
24             System.out.print("[Digite um valor inteiro entre " +
25             minimo + " e " + maximo + "] ");
26             valor_lido = teclado.nextInt();
27         } while (valor_lido < minimo || valor_lido > maximo);
28         return valor_lido;
29     }
30
31     // Metodo estatico privado para ler numero real
32     private static double ler_real() {
33         System.out.print("[Digite um valor real] ");
34         double valor_lido = teclado.nextDouble();
35         return valor_lido;
36     }
37
38     // Metodo estatico privado para ler numero real com restricao (
39     // nao zero)
40     private static double ler_real_diferente_de_zero() {
41         double valor_lido;
42         do {
43             System.out.print("[Digite um valor real diferente de zero
44             ] ");
45             valor_lido = teclado.nextDouble();
46         } while (valor_lido == 0);
47     }
48 }
```

```

40         return valor_lido;
41     }
42
43     // Metodo estatico privado que executa a logica baseada na
operacao escolhida
44     private static void realizar_calculo(int operacao) {
45         double resultado = 0;
46         switch (operacao) {
47             case 1:
48                 resultado = Calculadora.somar(ler_real(), ler_real())
;
49                 break;
50             case 2:
51                 resultado = Calculadora.subtrair(ler_real(), ler_real
());
52                 break;
53             case 3:
54                 resultado = Calculadora.multiplicar(ler_real(),
ler_real());
55                 break;
56             case 4:
57                 resultado = Calculadora.dividir(ler_real(),
ler_real_diferente_de_zero());
58                 break;
59         }
60         System.out.println("Resultado: " + resultado);
61     }
62
63     // Metodo estatico privado para exibir o menu
64     private static int menu() {
65         System.out.println("\nOpcoes:");
66         System.out.println("1. Somar dois numeros");
67         System.out.println("2. Subtrair um numero de outro");
68         System.out.println("3. Multiplicar dois numeros");
69         System.out.println("4. Dividir dois numeros");
70         System.out.println("0. Sair do aplicativo");
71         return ler_int(0, 4);
72     }
73
74     // Ponto de entrada
75     public static void main(String[] args) {
76         System.out.println("Bem-vindo ao aplicativo de calculo!");
77         boolean usuario_interagindo = true;
78
79         do {
80             int opcao = menu();
81             if (opcao != 0) {
82                 realizar_calculo(opcao);
83             } else {
84                 usuario_interagindo = false;
85             }

```

```
86         } while (usuario_interagindo);  
87  
88         System.out.println("Ate a proxima!");  
89     }  
90 }
```

Listing 3: Código-Fonte Completo: Sistema Calculadora